Multi-Agent Memory Blueprint v2

Multi-Agent Memory Blueprint v2

1. Purpose and Design Goals

Blueprint v2 turns the v1 architecture into an operational memory spec for a multi-agent system built on: - **OpenClaw** for runtime, sessions, and messaging - **Paperclip** for organizational structure, governance, and task orchestration - **Supermemory** for memory storage, retrieval, and recall primitives

Core goals

- prevent shared-memory pollution
- preserve agent specialization without breaking collaboration
- separate active context from historical memory
- make promotion, retrieval, and archive behavior enforceable
- support scale toward 50–100 agents without coordination collapse

Primary failure modes this design tries to prevent

- flat noisy memory buckets
- stale memories resurfacing as active truth
- project residue leaking into future work
- private memory becoming a shadow system
- unrestricted archive retrieval polluting active execution
- governance becoming more elaborate than retrieval effectiveness

---

2. System Roles and Operating Model

Orchestrator

The orchestrator is the top-level coordination authority.

Can: - create shared memory spaces - grant/revoke access - approve promotions into higher scopes - finalize project archive state - inspect private/restricted scopes when policy allows and audit it

Operators / Managers / Operatives

Privileged operational roles with assigned but not blanket access.

Can: - access assigned company/project memories - access cold storage when authorized - help curate durable knowledge - read governance only if designated as operatives

Agents

Execution principals.

Can: - use private memory - use assigned project memory - use explicitly granted company/restricted memory - not create shared spaces - not access cold storage by default

System Services

Named automated principals used for maintenance, indexing, summaries, deduplication, or audits. They must be treated as principals and logged.

---

3. Memory Hierarchy and Scope Definitions

0. Global Governance

System-wide rules, coordination laws, permanent standards. - container: `gov_global` - read: orchestrator + designated operatives - write: orchestrator only

1. Company Memory

Durable domain knowledge shared across projects in the same company. - examples: `company_web`, `company_games` - stores reusable workflows, stable domain standards, persistent operational patterns

2. Project Shared Memory

Live collaboration memory for active projects. - examples: `project_supermemory_fork` - stores blockers, decisions, constraints, handoff-critical state

3. Restricted Shared Memory

Limited-scope shared memory for named groups only. - examples: `restricted_billing`, `restricted_admin` - no automatic inheritance from project/company

4. Agent Private Memory

Local durable working memory for an individual agent. - examples: `agent_orchestrator_private` - stores heuristics, scratch patterns, unfinished local thinking worth keeping temporarily

5. Agent Experience

Curated long-term specialization knowledge. - primarily markdown experience files - stores lessons, recurring bottlenecks, anti-patterns, repeatable tactics

6. User Memory

User-specific persistent preferences and facts. - example: `user_<id>`

7. Session Memory

Active thread/session continuity. - keyed by `conversationId`

8. Ephemeral Context

Very short-lived context that should usually die quickly unless promoted.

9. Cold Storage

Archived completed project memory. - examples: `archive_project_supermemory_fork` - historical, not routine retrieval - accessible by orchestrator and authorized operators/managers only

---

4. Access Control and ACL Semantics

Core ACL Model

Access is evaluated against: - subject - resource - action - context

Default posture: **deny unless explicitly allowed**.

Permission Verbs

- `read`
- `write`
- `update`
- `append`
- `promote`
- `approve`
- `archive`
- `restore`

- `grant`
- `revoke`
- `audit`
- `delete`

`write` does not imply `promote`, `archive`, `grant`, or `delete`.

Grant Types

- direct grant
- role grant
- assignment grant
- delegated temporary grant
- break-glass grant

Precedence Rules

1. hard system deny
2. explicit deny
3. break-glass allow if valid and not blocked by hard deny
4. explicit allow
5. delegated/role/assignment allow
6. inherited allow where supported
7. otherwise deny

Canonical ACL Rules

- deny beats allow
- restricted/private/user/cold storage do not inherit by default
- project assignment grants project access only
- company membership may grant company access where policy allows
- session participation grants session access only
- archived resources stop inheritance and require explicit archive authorization
- all sensitive reads and ACL mutations are audited

Sensitivity vs Visibility

These are separate. - **visibility** = who may access - **sensitivity** = how risky the content is if mishandled

Sensitivity values: - `public_safe` - `internal` - `restricted` - `secret`

---

5. Write Rules and Memory Hygiene

What to write

- verified findings
- confirmed blockers

- decisions
- reusable lessons
- handoff-critical state
- durable company methods
- stable user facts/preferences

What not to write

- raw chatter
- duplicate updates
- speculative noise without labeling
- verbose logs when concise summaries exist
- stale project residue
- unbounded tool output
- secrets copied unnecessarily into broader scopes

Compression Rule

Before anything is promoted or broadly stored, compress it into one of: - fact - decision - constraint - policy - playbook - lesson - anti-pattern - handoff summary

Canonical hygiene rules

1. one fact/decision per item when possible
2. explicit entity names and IDs
3. set effective time windows when facts change
4. use supersession links on updates
5. mark uncertainty explicitly
6. prefer curated summaries over raw transcripts
7. attach ACL tags at write time
8. do not store speculative filler as durable memory

---

6. Promotion Model

Core Principle

Promotion is a **lossy distillation step**. Higher layers must contain: - less volume - higher confidence - broader reuse value - lower project-specific residue - clearer ownership

Default Promotion Path

- `ephemeral/session -> agent_private`
- `agent_private -> project`
- `project -> company`
- `company -> governance`

Side Paths

- `project -> experience`
- `agent_private -> experience`
- `user/session -> user`
- `project -> cold_storage` through closeout only

Promotion Gates

A memory should be promoted only if it passes: 1. **Truth/Confidence gate** 2. **Reuse gate** 3. **Scope-fit gate** 4. **Compression gate** 5. **Ownership gate**

Suggested thresholds

- to `project`: confidence >= `0.65`
- to `company`: confidence >= `0.80`
- to `governance`: confidence >= `0.90`

Promotion Decision Matrix

Promote immediately

- confirmed project decisions
- confirmed blockers with owner/next step
- handoff-critical context
- stable user preference updates

Promote after review

- company standards
- governance rules
- restricted shared memory
- access-changing assumptions
- anything with unresolved counterevidence

Never promote directly

- raw transcripts
- temporary debug spam
- tool output without interpretation
- speculative unlabeled claims
- unnecessary secrets in broader scopes

Promotion Metadata

Every promoted item should include: - `promoted_from` - `promotion_state` - `promotion_reason` - `approved_by` - `derived_from` - `review_status`

---

7. Retrieval Strategy

Retrieval principle

Scope before similarity. The system should search the most relevant layer first, not the largest one.

Retrieval defaults by scenario

Active project execution

1. session
2. ephemeral
3. project
4. restricted
5. user
6. company
7. governance
8. experience

Cold storage disabled.

Policy/compliance question

1. governance
2. restricted
3. company
4. project

User personalization

1. user
2. session
3. project if task-specific

“How did we solve this before?”

1. project
2. experience
3. company
4. cold storage

Self-improvement / agent operations

1. private
2. experience
3. project/company as relevant

Hard Filters Before Ranking

- ACL visibility
- relevant project/company/user/session IDs
- allowed status (`active` by default)
- effective time-window validity
- exclude archived by default
- exclude superseded by default

Ranking Model

```
final_score =
  (0.35 * semantic_similarity)
+ (0.20 * scope_relevance)
+ (0.15 * authority_score)
+ (0.10 * freshness_score)
+ (0.08 * confidence_score)
+ (0.05 * importance_score)
+ (0.04 * verification_score)
+ (0.03 * quality_score)
- stale_penalty
- superseded_penalty
- contradiction_penalty
- noise_penalty
```

Canonical Ranking Rules

1. Never retrieve archived or superseded memories by default.
2. Project memory outranks company memory for active project work.
3. Governance outranks everything for rules and safety.
4. User-confirmed preferences outrank inferred preferences.
5. Curated summaries outrank raw logs.
6. Contradictions require explicit winner/loser metadata.
7. Ephemeral memory dies quickly unless promoted.
8. Completed project memory moves to cold storage and stops polluting normal recall.

---

8. Lifecycle Management

Core Principle

Active memory should be useful now. Historical memory should not pollute routine retrieval.

Lifecycle State Machine

```
proposed -> approved -> active -> stale -> archived
                \-> rejected
```

```
active -> superseded
active -> disputed -> resolved
```

TTL / Review Defaults

Governance

- persistent
- no TTL
- review every 90–180 days

Company

- persistent
- review every 60–120 days
- outdated standards get superseded or archived

Project

- lifecycle-bound
- soft stale after 14 days inactive
- archive on completion/cancellation/30 days inactive

Restricted Shared

- persistent or project-bound
- review every 30–90 days

Agent Private

- medium-term persistent
- stale after 30 days unused
- purge low-value notes after 90–180 days unless pinned or promoted

Experience

- persistent curated
- review every 60–120 days

User

- stable facts: persistent with periodic verification
- preferences: review every 90–180 days
- transient circumstances: 7–30 days unless reaffirmed

Session

- active conversation + 24–72h grace

Ephemeral

- minutes to task duration
- purge after task completion unless promoted

Cold Storage

- archival only
- excluded from default retrieval

Stale Rule

A memory becomes stale when it is old, unverified, tied to a completed project, repeatedly outranked, or no longer relevant to active execution.

---

9. Project Closeout Protocol

Project closeout is mandatory and must be a named workflow.

Trigger states

- `completed`
- `cancelled`
- `paused_long_term`
- `superseded`

Mandatory checklist

1. project marked closing
2. active memory frozen
3. bottlenecks extracted
4. durable lessons reviewed
5. specialist `experience.md` updated
6. archive package created
7. access revoked/reduced
8. archive manifest written
9. low-value active-scope residue sanitized

Closeout phases

Phase 1: Freeze

- mark project memory space `closing`
- stop normal writes except closeout notes

Phase 2: Distill

Create a closeout summary containing: - project outcome - deliverables complete/incomplete - final architecture/decision summary - unresolved risks - blockers encountered - reusable lessons - candidate promotions - archive sensitivity notes

Phase 3: Promote

Extract: - stable patterns -> company - durable global coordination rules -> governance - specialist lessons -> experience - user facts/preferences -> user

Phase 4: Archive Package

Archive: - final summary - decision index - deliverables index - unresolved issues list - source refs / artifact links - access and sensitivity manifest

Phase 5: Revoke / Reduce Access

- remove routine agent access
- keep orchestrator access
- keep operator/manager access per policy

Phase 6: Sanitize Active Scopes

- remove obsolete project refs from active retrieval defaults
- downgrade project memory to archived
- clear low-value temporary private residues tied only to that project
- preserve extracted experience entries

Closeout extraction limits

- max 5 durable lessons per specialist per project
- max 10 archive tags
- max 1 short archive summary
- max 3 company-promotion candidates

---

10. Experience System

What experience is

A curated specialist layer for: - repeated bottlenecks - reliable tactics - failure signatures - heuristics worth carrying forward

What experience is not

- raw project memory
- generalized archive dumping
- diaries or long project narrative

Experience entry format

```
## <lesson title>
- pattern:
- signal:
- interpretation:
- action:
- boundaries:
- evidence:
- source_projects:
- confidence:
- last_validated_at:
- promotion_candidate: yes|no
```

Experience extraction rules

- write lessons, not diaries
- merge duplicates into stronger entries
- include boundaries / when not to apply
- keep evidence traceable
- mark company-promotion candidates only if genuinely generalizable

---

11. Metadata Schema

Required v2 themes

All important durable memory should support: - lineage - review state - contradiction handling - TTL/lifecycle state - sensitivity/visibility separation - retrieval hints - provenance strength

Important fields

- `schema_version`
- `memory_id`
- `canonical_key`
- `memory_type`
- `memory_scope`
- `visibility`
- `sensitivity`
- `status`
- `confidence`
- `review_status`
- `verification_state`
- `promoted_from`
- `promotion_state`
- `promotion_reason`
- `derived_from`
- `supersedes`
- `superseded_by`

- `contradiction_set`
- `effective_from`
- `effective_until`
- `ttl`
- `expires_at`
- `last_validated_at`
- `retrieval_priority`
- `quality_score`
- `dedup_key`
- `timestamp`

Canonical JSON schema file: - `agents/METADATA_SCHEMA_V2.json`

---

12. Registries and Filesystem Layout

Existing registries

- `agents.json`
- `companies.json`
- `projects.json`
- `memory_spaces.json`

New registries recommended

- `promotions.json`
- `archives.json`
- `experience_index.json`
- optionally `policies.json`

File layout additions

- `agents/ACL_POLICY.md`
- `agents/PROMOTION_RULES.md`
- `agents/RETRIEVAL_POLICY.md`
- `agents/LIFECYCLE_POLICY.md`
- `agents/CLOSEOUT_PROTOCOL.md`
- `agents/CONTRADICTION_POLICY.md`
- `agents/METADATA_SCHEMA_V2.json`

---

13. Naming Conventions and Canonical Keys

Examples: - `gov_global` - `company_web` - `project_supermemory_fork` - `restricted_billing` - `agent_orchestrator_private` - `archive_project_supermemory_fork`

Canonical keys should describe durable meaning rather than raw source text. Example: - `project_supermemory_fork:deployment:railway-timeout-rule`

14. Operational Examples

Example promotion

A verified deployment workaround appears repeatedly on web projects. - starts in project memory - gets compressed into a reusable rule - promoted to company memory with approval - linked to source evidence

Example contradiction update

An older company standard is replaced by a newer approved version. - new memory gets `supersedes` - old memory gets `superseded_by` - retrieval shows only the new one by default

Example closeout bundle

At project end: - final summary written - lessons extracted to experience - archive manifest created - default retrieval revoked - project memory moved to cold storage

---

15. Initial Implementation Plan

Minimum viable v2

1. finalize v2 documents
2. adopt metadata v2 fields
3. add promotion and closeout registries
4. enforce default deny ACLs
5. exclude archived/superseded memory by default
6. implement project closeout workflow
7. make cold storage non-routine

What can remain manual at first

- promotion review
- archive manifest writing
- experience extraction
- contradiction resolution

What should later become automated

- dedup checks
- stale detection
- archive packaging
- promotion candidate scoring
- ACL audit analysis

One-Sentence v2 Summary

Blueprint v2 defines a scoped, reviewable memory lifecycle in which active project context is distilled into durable experience, company standards, and rare governance rules, while completed project memory is frozen, packaged, and moved to cold storage instead of leaking into routine recall.